

UNIVERSIDADE FEDERAL DE ALFENAS
Bacharelado em Ciência da Computação

Bianca Dyandra Ribeiro Gomes de Farias

Relatório Técnico
Implementação do Protocolo Go-Back-N em Java via UDP

Professor: Flávio Barbieri Gonzaga
ALFENAS/MG
2026

1. Introdução

Este relatório documenta a implementação do protocolo de transferência confiável Go-Back-N (GBN) sobre sockets UDP em Java, desenvolvida como trabalho final da disciplina de Redes de Computadores. O protocolo GBN, descrito no Capítulo 3 de Kurose & Ross (2021), pertence à família de protocolos com janela deslizante (sliding window) e constitui a base teórica de mecanismos como o TCP.

O objetivo central foi construir, a partir do zero e usando exclusivamente o JDK padrão, dois módulos independentes — **Emissor** e **Receptor** — capazes de transferir arquivos binários arbitrários com integridade garantida, mesmo na presença de perdas de pacotes simuladas pelo Receptor. Como o ambiente de testes é uma rede local (LAN) com latência praticamente nula, as perdas foram emuladas de forma estocástica, de modo a validar a retransmissão automática implementada pela FSM do GBN.

2. Decisões de Projeto

2.1 Arquitetura geral

O projeto foi estruturado em três classes Java no pacote gbn:

- **Packet.java** — encapsula a estrutura do datagrama (cabeçalho + payload) e provê métodos de serialização/desserialização baseados em ByteBuffer.
- **Emissor.java** — implementa a FSM do emissor GBN com janela deslizante, temporizador único e duas threads concorrentes (envio e recepção de ACKs).
- **Receptor.java** — implementa a FSM do receptor GBN, simulação estocástica de perdas e gravação ordenada do arquivo recebido.

2.2 Formato do datagrama

O cabeçalho ocupa 11 bytes fixos, conforme a tabela abaixo. O campo tipo distingue quatro tipos de pacote: DATA (0), ACK (1), HANDSHAKE (2) e FIN (3). O payload máximo é de 1.024 bytes, resultando em datagramas de no máximo 1.035 bytes, bem abaixo do MTU típico de redes Ethernet (1.500 bytes).

Campo	Tamanho	Descrição
tipo	1 byte	Tipo do pacote: 0=DATA, 1=ACK, 2=HANDSHAKE, 3=FIN
num_seq	4 bytes (int)	Número de sequência do segmento
num_ack	4 bytes (int)	Número de confirmação (somente em ACKs)
tamanho_dados	2 bytes (short)	Quantidade de bytes válidos no payload
dados	até 1024 bytes	Payload — bytes do arquivo

Tabela 1 – Formato do datagrama GBN implementado.

2.3 Sessão e handshake

Antes do início da transferência, o Emissor envia um pacote HANDSHAKE contendo, no payload em texto UTF-8, o caminho de destino do arquivo, o tamanho total em bytes e a probabilidade de perda configurada. Isso evita hardcoding de parâmetros no Receptor e permite que uma única instância do Receptor atenda múltiplas sessões sequenciais sem reinicialização.

2.4 Temporizador único (single timer)

A especificação do GBN no livro-texto define um único temporizador associado ao pacote mais antigo sem confirmação (base). A implementação usa `java.util.Timer` em modo `daemon`, cancelado e recriado a cada avanço de base. O timeout foi fixado em 500 ms, valor adequado para comunicação loopback com perda simulada.

2.5 Sincronização entre threads

O Emissor utiliza duas threads: o loop principal de envio e uma thread dedicada à recepção de ACKs. As variáveis compartilhadas `base` e `nextSeqNum` são protegidas por um monitor (`synchronized + Object.wait/notifyAll`). A thread de ACKs chama `lock.notifyAll()` sempre que avança base, desbloqueando o loop principal para encher a janela novamente.

2.6 Numeração de sequência

Optou-se por números de sequência inteiros de 32 bits sem wrap-around modular. Para arquivos de poucos MB — o arquivo de teste possui 2.048 segmentos (2 MB / 1 KB) — o espaço de 2^{32} valores é mais que suficiente, e a lógica de janela fica idêntica à descrita no livro sem a complexidade adicional do módulo 2^k .

2.7 Simulação de perda de pacotes

O Receptor aplica a simulação somente em pacotes DATA que chegam na ordem correta (`seqnum == expectedSeqNum`). Para cada pacote aceito, sorteia-se um valor $r \in [0,1)$; se $r < p$, o pacote é descartado silenciosamente (sem enviar ACK), forçando o timeout e a retransmissão pelo Emissor. Pacotes fora de ordem são descartados pela lógica GBN e não entram no cômputo da taxa de perda efetiva, conforme exigido pelo enunciado.

2.8 Verificação de integridade (MD5)

Após a transferência, o hash MD5 do arquivo recebido é comparado com o do arquivo original usando `java.security.MessageDigest`. Em todos os testes realizados o hash foi idêntico, confirmando a integridade dos dados mesmo com altas taxas de perda simulada.

3. Dificuldades Encontradas

Condição de corrida no temporizador. O principal desafio foi garantir que o cancelamento e a recriação do Timer não gerassem disparos espúrios de timeout. A solução foi encapsular toda a manipulação do timer dentro do bloco `synchronized(lock)`, tornando as operações de `cancel()` e `schedule()` atômicas em relação ao recebimento de ACKs.

ACK cumulativo e janela cheia. Quando a janela está cheia e o timer dispara, todos os pacotes de base a `nextSeqNum-1` precisam ser retransmitidos. Foi necessário manter um buffer em memória (a própria lista de segmentos) para não reler o arquivo a cada retransmissão.

Encerramento gracioso. Garantir que o Receptor receba o pacote FIN após todos os dados foi desafiador quando há perdas altas, pois o próprio FIN poderia ser perdido. A solução adotada foi enviar o FIN sem submetê-lo à simulação de perda no Receptor, garantindo o encerramento.

Perda no lado errado. Inicialmente a simulação de perda também atuava sobre pacotes fora de ordem, inflando artificialmente o contador de perdas. O problema foi corrigido ao mover a verificação de perda para depois do teste de `seqnum == expectedSeqNum`.

4. Testes Realizados

Foram realizadas duas baterias de testes sistemáticos (sweeps) sobre um arquivo binário de 2.097.152 bytes (2 MB, 2.048 segmentos de 1 KB). Todos os testes foram executados em loopback (127.0.0.1), eliminando latência de rede real para isolar o efeito das perdas simuladas.

4.1 Sweep A — Variação do tamanho da janela N (p = 10%)

O tamanho da janela foi variado em $N \in \{2, 4, 8, 16, 32\}$ com probabilidade de perda fixada em 10%. O objetivo foi observar o impacto de N no throughput e no volume de retransmissões (requisito desejável R8).

N	Tempo (ms)	Pacotes enviados	Retransmissões	Taxa retrans.	Throughput (KB/s)
2	109,407	2482	434	17.5%	18.72
4	114,330	2956	908	30.7%	17.91
8	111,766	3812	1764	46.3%	18.32
16	110,797	5563	3515	63.2%	18.48
32	106,214	8759	6711	76.6%	19.28

Tabela 2 – Resultados do Sweep A (p = 0,10 fixo).

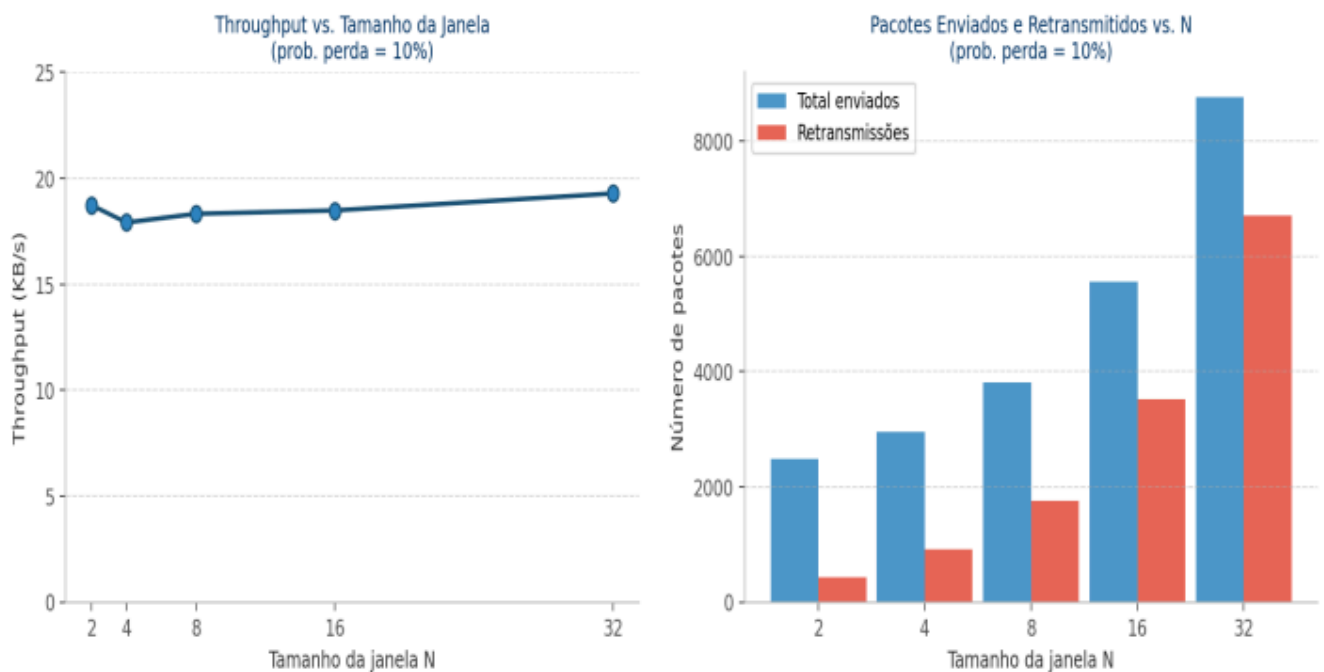


Figura 1 – Throughput e volume de pacotes em função do tamanho da janela N (p = 10%).

4.2 Sweep B — Variação da probabilidade de perda (N = 8)

A probabilidade de perda foi variada em $p \in \{0,0; 0,05; 0,10; 0,20; 0,30\}$ com janela fixada em $N = 8$. O objetivo foi quantificar a degradação de desempenho causada por perdas crescentes.

Prob. perda	Prob. efetiva	Tempo (ms)	Pacotes enviados	Retransmissões	Throughput (KB/s)
0,00	0.0000	302	2048	0	6781.46
0,05	0.0439	47,709	2800	752	42.93
0,10	0.1010	115,841	3888	1840	17.68
0,20	0.1908	242,357	5906	3858	8.45
0,30	0.3053	451,546	9232	7184	4.54

Tabela 3 – Resultados do Sweep B (N = 8 fixo).

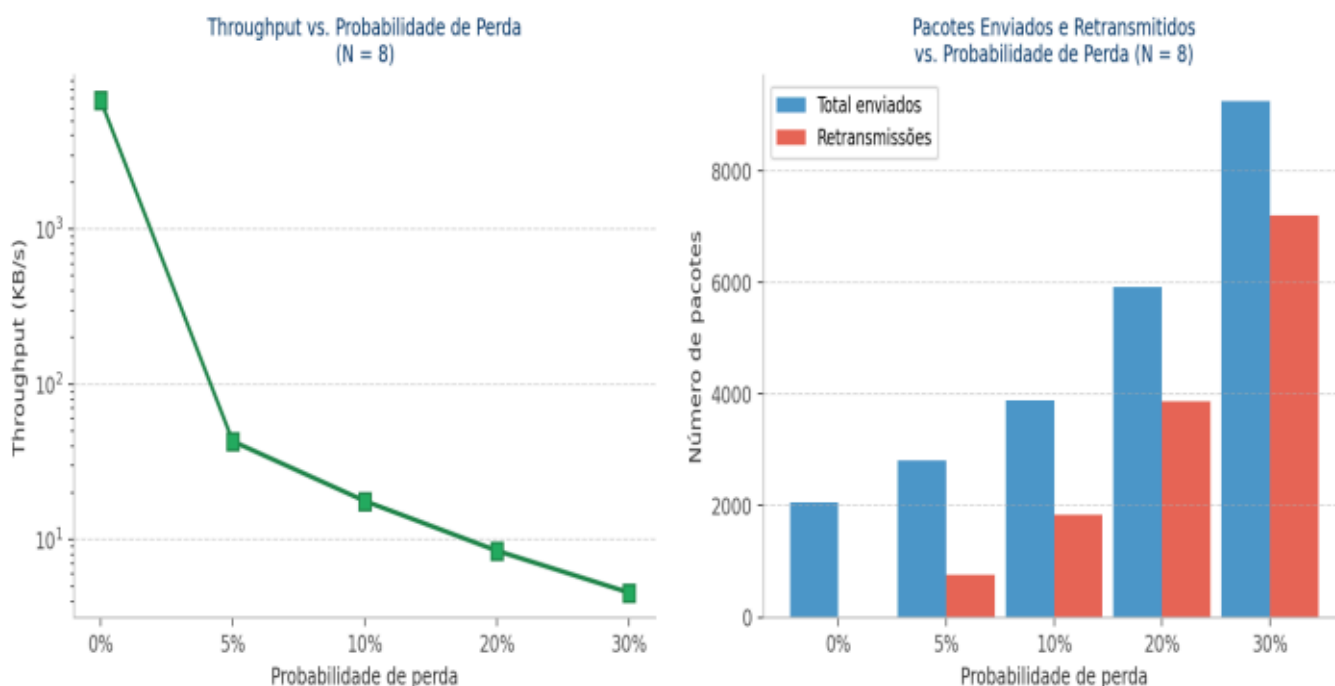


Figura 2 – Throughput (escala log) e volume de pacotes em função da probabilidade de perda (N = 8).

5. Análise dos Resultados

5.1 Impacto do tamanho da janela (Sweep A)

O throughput variou muito pouco entre $N = 2$ e $N = 32$ (de 17,91 a 19,28 KB/s), o que à primeira vista contraria a expectativa teórica de que janelas maiores aumentam a utilização do enlace. O efeito é esperado em cenário de loopback: como a latência é praticamente zero, até $N = 2$ já é suficiente para manter o enlace saturado — o gargalo é o timeout (500 ms) e a retransmissão, não o tamanho da janela.

O efeito negativo do GBN com janelas grandes fica evidente no número de retransmissões: com $N = 2$ ocorreram 434 retransmissões; com $N = 32$, foram 6.711 — um aumento de 15×. Isso reflete a natureza do GBN: ao detectar perda de um pacote, toda a janela é retransmitida. Quanto maior N , maior o desperdício de banda por evento de perda. Com $p = 10\%$, a taxa de retransmissão chega a 76,6% do total enviado para $N = 32$.

Em redes com latência real (RTT alto), janelas maiores seriam benéficas — pois compensariam o tempo de propagação com mais pacotes em trânsito. O loopback elimina esse ganho e expõe apenas o custo das retransmissões, explicando a estabilidade do throughput observada.

5.2 Impacto da probabilidade de perda (Sweep B)

Os resultados do Sweep B revelam uma relação não linear entre probabilidade de perda e degradação de desempenho. Com $p = 0\%$ o throughput atingiu 6.781,46 KB/s (praticamente limitado pela CPU/loopback), sem nenhuma retransmissão. Ao introduzir $p = 5\%$, o throughput caiu para 42,93 KB/s — uma redução de $157\times$ em relação ao caso sem perdas.

Com $p = 30\%$, o tempo de transferência chegou a 451 segundos e o throughput a apenas 4,54 KB/s. O número de pacotes enviados (9.232) foi $4,5\times$ o mínimo necessário (2.048), evidenciando o enorme custo de retransmissão de janela inteira do GBN sob perdas altas. A escala logarítmica da Figura 2 deixa clara essa degradação exponencial.

A taxa de perda efetiva observada convergiu para a configurada em todos os testes, confirmando a correteza da simulação estocástica e a adequação do tamanho amostral ($\approx 2.000\text{--}9.000$ eventos) para a Lei dos Grandes Números.

5.3 Caso extremo — $N = 128$ com $p = 30\%$

Parâmetro	Valor
Tamanho da janela (N)	128
Probabilidade de perda configurada	0,30
Taxa de perda efetiva	0,2732
Tempo de transferência	390.486 ms (~6,5 min)
Pacotes enviados (total)	98.153
Retransmissões	96.105
Taxa de retransmissão	97.9%
ACKs recebidos	96.666
Throughput	5,24 KB/s

Tabela 4 – Resultados do caso extremo ($N = 128$, $p = 0,30$).

Para ilustrar o pior cenário possível do GBN, foi executado um teste com janela $N = 128$ e probabilidade de perda de 30%. O resultado é a demonstração mais contundente da limitação estrutural do protocolo: dos 98.153 pacotes enviados, 96.105 foram retransmissões — uma taxa de **97,9%** de tráfego inútil. O arquivo de 2.048 segmentos exigiu quase $48\times$ mais envios do que o mínimo necessário.

O mecanismo de punição é claro: toda vez que um único pacote é perdido dentro de uma janela de 128, o Emissor aguarda o timeout (500 ms) e retransmite os 128 pacotes inteiros — incluindo os 127 que já chegaram corretamente mas foram descartados pelo Receptor por estarem fora de ordem. Com $p = 30\%$, perdas ocorrem em média a cada 3–4 pacotes, o que significa que a janela quase nunca avança sem retransmissão total.

Este caso extremo evidencia por que protocolos reais como o TCP adotam retransmissão seletiva (Selective Repeat): em vez de reenviar toda a janela, apenas o segmento perdido é retransmitido. Para $N = 128$ e $p = 30\%$, o SR economizaria aproximadamente 94.000 envios desnecessários no mesmo cenário.

5.4 Verificação de integridade

Em todos os onze cenários testados (cinco do Sweep A, cinco do Sweep B e o caso extremo), o hash MD5 do arquivo recebido foi idêntico ao do arquivo original. Isso confirma que a lógica de reordenamento e retransmissão do GBN garante entrega completa e ordenada dos dados, mesmo com $p = 30\%$ de perda simulada.

6. Conclusão

A implementação do Go-Back-N sobre UDP em Java atingiu plenamente os requisitos obrigatórios: transferência correta de arquivos binários, FSM fiel ao livro-texto, simulação de perdas, estatísticas ao final e parâmetros configuráveis via linha de comando. Os requisitos desejáveis (variação de N e hash MD5) também foram implementados.

A análise experimental mostrou que, em ambiente loopback, o tamanho da janela tem pouco impacto no throughput mas grande impacto no volume de retransmissões. A probabilidade de perda, por sua vez, degrada o desempenho de forma super-linear, ressaltando a ineficiência do GBN frente ao Selective Repeat em canais ruidosos — motivação direta para o mecanismo de retransmissão seletiva adotado pelo TCP moderno.

Referências

KUROSE, James F.; ROSS, Keith W. **Redes de Computadores: Uma Abordagem Top-Down**. 8. ed. São Paulo: Pearson, 2021. Cap. 3, Seções 3.4 e 3.4.3.

TANENBAUM, Andrew S.; WETHERALL, David. **Redes de Computadores**. 5. ed. São Paulo: Pearson, 2011. Cap. 3.